DAKTİLO.JS (TYPEWRITER ENGINE)

Web İçin Sıfır Bağımlılıklı Daktilo Estetiği, Organik Mürekkep Simülasyonu ve Web Audio Ses Motoru

ONAYLANDI // SÜRÜM 1.0.0-RC1

ÜRÜN ADI	`daktilo-js` (NPM & CDN)	**DOKÜMAN TARIHI**	22 Ekim 2026
PAKET TÜRÜ	ESM / CJS / IIFE (TypeScript)	**HEDEF BOYUT**	< 5 KB (Minified + Gzipped)
BAĞIMLILIK	Sıfır (Zero-Dependency)	**DURUM**	Mimari Tasarım & MVP

1. YÖNETICI ÖZETI VE ÜRÜN VIZYONU

Modern web ekosisteminde metin odaklı siteler, dijital bahçeler (digital gardens), kişisel bloglar ve edebi yayınlar standartlaşmış kurumsal şablonlardan uzaklaşıp **"kusurlu, dokunsal ve analog"** deneyimlere yönelmektedir. Mevcut JavaScript ekosisteminde bulunan `Typed.js` veya `TypeIt` gibi kütüphaneler yalnızca metni harf harf ekrana yazdırma animasyonuna odaklanmakta; tipografik doku, kağıt estetiği, mekanik mürekkep dağılması ve donanımsal ses simülasyonunu bütüncül olarak sunmamaktadır.

Daktilo.js, herhangi bir HTML kapsayıcısını (veya tüm web sayfasını) tek bir yapılandırma nesnesiyle gerçekçi bir daktilo kağıdına dönüştüren, açık kaynaklı ve sıfır bağımlılıklı bir mikro-kütüphanedir. Harici ses dosyalarına (MP3/WAV) ihtiyaç duymadan tarayıcının yerleşik `Web Audio API`'sini kullanarak gerçek zamanlı mekanik tuş ve satır başı çanı (carriage return bell) sesleri sentezler; DOM üzerindeki metin düğümlerine mikro-hiza kaymaları ve mürekkep yoğunluğu varyasyonları uygular.

< 5 KB	**0 MP3**	**60 FPS**	**%100 TS**
GZIP PAKET BOYUTU	PROSEDÜREL WEB AUDIO SESI	DOM & CSS PERFORMANSI	TAM TIP GÜVENLIĞI

2. PROBLEM TANIMI VE PAZAR BOŞLUĞU

- **Yüzeysel Animasyon Kütüphaneleri:** Mevcut kütüphaneler yalnızca string manipülasyonu yapar. Geliştiriciler gerçek bir daktilo teması oluşturmak istediklerinde font yükleme, kağıt dokusu, mürekkep kusurları ve ses dosyalarının senkronizasyonunu sıfırdan kodlamak zorunda kalmaktadır.
- **Harici Ses Dosyası Yükü (Asset Bloat):** Tuş sesi eklemek isteyen projeler harici `.mp3` veya `.wav` dosyaları yüklemekte, bu durum CDN yolları (path resolution), ağ gecikmesi ve tarayıcı önbellek sorunlarına yol açmaktadır.
- **Dijital Sterillik:** Bilgisayar fontları her harfi kusursuz aynı opaklık ve hizada basar. Oysa gerçek bir daktiloda şerit mürekkebi düzensizdir ve mekanik çekiçler milimetrik dikey sapmalarla kağıda vurur.

3. HEDEF KITLE VE KULLANIM SENARYOLARI

KULLANICI PROFILI	KULLANIM SENARYOSU	BEKLENEN TEMEL DEĞER
Bağımsız Yazarlar & Blog Sahipleri	Astro, Hugo, Ghost veya WordPress tabanlı kişisel not/ deneme siteleri.	Tek satır CDN koduyla tüm makaleleri saman kağıdı ve daktilo şeridi görünümüne kavuşturmak.
Frontend Geliştiriciler	React, Vue, Next.js projelerinde portfolyo, "Hakkımda" veya interaktif hikaye sayfaları.	Tree-shakeable ESM paketi, TypeScript desteği ve SSR (Server-Side Rendering) uyumluluğu.
Oyun & Hikaye Tasarımcıları	Dedektiflik, retro-arşiv, SCP/ gizli evrak temalı web tabanlı metin oyunları.	Otomatik evrak damgası (metadata), gerçekçi yazım hatası (overstrike) ve mekanik ses motoru.

4. FONKSIYONEL GEREKSINIMLER VE ÇEKIRDEK MODÜLLER

4.1. Modül 1: Tema ve Kağıt Motoru (Paper & Theme Engine)

Kütüphane başlatıldığında hedef DOM elemanına izole edilmiş CSS değişkenleri (Custom Properties) ve gerekli Google Fontlarını (`Special Elite` ve `Courier Prime`) otomatik enjekte eder.

- **Hazır Kağıt Ön Ayarları (Presets):**
  - `vintage`: Sararmış saman kağıdı (`#F4F1EA`), koyu gri-siyah şerit mürekkebi (`#262421`), kırmızı düzeltme şeridi (`#C84B31`) ve CSS radial-gradient tabanlı mikro kağıt gözenekleri.
  - `clean`: Modern beyaz kağıt (`#FAFAFA`), keskin siyah mürekkep (`#141414`), mavi şerit vurgusu (`#1D4ED8`).
  - `carbon`: Karanlık mod (Dark Mode) uyumlu karbon kağıdı (`#18181B`), gümüşi/beyaz mürekkep (`#E4E4E7`), kehribar vurgular (`#F59E0B`).
- **Evrak Üst Bilgisi ve Damga (Stamp Metadata):** İsteğe bağlı olarak metin bloğunun üst kısmına tarih, dosya numarası, gizlilik derecesi ve eğimli (rotated) "ARŞİV / TASLAK" damgası ekler.

4.2. Modül 2: Organik Mürekkep Simülasyonu (Organic Ink Jitter)

DOM içerisindeki metin düğümleri `TreeWalker` API'si ile taranır (HTML etiketlerine ve event listener'lara zarar verilmez). Belirlenen `inkWear` (0.0 - 1.0) oranına göre karakterler rastgele seçilerek mikro-transformasyonlara uğratılır:

- **Dikey Sapma (Y-Offset):** `-0.6px` ile `+0.6px` arasında rastgele dikey kayma.
- **Mürekkep Yoğunluğu (Opacity):** Şeridin eskimesini simüle eden `0.72` ile `1.0` arası opaklık.
- **Çift Vuruş / Dağılma (Ink Bleed):** Çok düşük olasılıkla (`%3`) harfe `0.4px` bulanık `text-shadow` eklenerek sert tuş vuruşu hissi verilir.

4.3. Modül 3: Harf Harf Yazım ve "Overstrike" (Hata Düzeltme) Motoru

Klasik yazım kütüphanelerinde hata yapıldığında karakter `Backspace` ile silinir. Ancak gerçek bir mekanik daktiloda geri silme tuşu kağıttaki mürekkebi silemez; yazar geri gider, yanlış harfin üzerine `x` veya `-` basar ve doğru harfle devam eder.

- **Gerçekçi Overstrike Modu:** `typoRate: 0.04` ayarlandığında motor %4 ihtimalle klavyede komşu bir tuşa basar, 150ms duraksar, yanlış harfin üzerini çizer (`<del>` veya `X` bindirmesi) ve doğru harfi yazar.
- **İnsan Ritim Varyasyonu (Human Cadence):** Sabit `50ms` gecikme yerine noktalama işaretlerinde (`. , ;`) duraksar, kelime içindeki ardışık harflerde hızlanır.

4.4. Modül 4: Prosedürel Web Audio Ses Centezleyici (Zero-Asset Audio)

Hiçbir harici ses dosyası indirilmeden `AudioContext` üzerinden 3 farklı mekanik ses sentezlenir:

1. **Tuş Vuruşu (Key Clack):** 25ms süreli White Noise tamponu (buffer), üstel sönümleme (exponential decay) ve her vuruşta `1100Hz - 1600Hz` arasında rastgele değişen Bandpass filtre ile mekanik çekiç sesi üretilir.
2. **Boşluk Tuşu (Spacebar Thud):** Daha tok ve pes frekanslı (`400Hz - 600Hz`) Lowpass filtreli kasa sesi.
3. **Satır Sonu Çanı (Carriage Return Ding):** Satır sonuna gelindiğinde veya ` karakterinde `2200Hz` frekansında Sine osilatörü ile metalik çan sesi çalınır.

5. TEKNIK MIMARI VE KOD ÖRNEKLERI

5.1. Geliştirici API Tasarımı ve TypeScript Arayüzü

```
// src/types.ts
export type PaperPreset = 'vintage' | 'clean' | 'carbon';

export interface StampMetadata {
  docNo?: string;
  date?: string;
  author?: string;
  classification?: string; // Örn: "TASLAK", "ARŞİV", "GİZLİ"
}

export interface DaktiloOptions {
  paper?: PaperPreset;          // Varsayılan: 'vintage'
  ribbonColor?: string;         // Varsayılan: '#c84b31' (Kırmızı şerit)
  maxWidth?: string;            // Varsayılan: '65ch'
  inkWear?: number;             // 0 (kapalı) ile 1 (yoğun eskime) arası. Varsayılan: 0.35
  animate?: boolean;            // Harf harf yazım animasyonu. Varsayılan: false
  speed?: number;               // Ortalama yazım hızı (ms). Varsayılan: 45
  typoRate?: number;            // Gerçekçi daktilo hatası oranı (0 - 0.2). Varsayılan: 0.02
  overstrikeChar?: 'x' | '-';   // Hata üzerine basılacak karakter. Varsayılan: 'x'
  sound?: boolean;              // Web Audio mekanik ses motoru. Varsayılan: false
  volume?: number;              // Ses seviyesi (0.0 - 1.0). Varsayılan: 0.4
  stamp?: StampMetadata | null; // Üst evrak damgası
}
```

5.2. Web Audio API ile Dosyasız Ses Motoru Uygulaması

```
// src/audio.ts
export class MechanicalSoundEngine {
```

```
  private ctx: AudioContext | null = null;
  private volume: number;
  private enabled: boolean;

  constructor(enabled = false, volume = 0.4) {
    this.enabled = enabled;
    this.volume = volume;
    if (typeof window !== 'undefined' && this.enabled) {
      const unlock = () => {
        if (!this.ctx) {
          const AudioCtx = window.AudioContext || (window as unknown as { webkitAudioContext: typeof
AudioContext }).webkitAudioContext;
          this.ctx = new AudioCtx();
        }
        if (this.ctx.state === 'suspended') this.ctx.resume();
      };
      window.addEventListener('pointerdown', unlock, { once: true });
      window.addEventListener('keydown', unlock, { once: true });
    }
  }

  public playKeyClick(isSpace = false): void {
    if (!this.enabled || !this.ctx) return;
    const now = this.ctx.currentTime;
    const duration = isSpace ? 0.045 : 0.028;
    const bufferSize = Math.floor(this.ctx.sampleRate * duration);
    const buffer = this.ctx.createBuffer(1, bufferSize, this.ctx.sampleRate);
    const data = buffer.getChannelData(0);

    for (let i = 0; i < bufferSize; i++) {
      // Mekanik darbe zarfı (Sharp attack, exponential decay)
      const env = Math.exp(-i / (this.ctx.sampleRate * (isSpace ? 0.012 : 0.006)));
      data[i] = (Math.random() * 2 - 1) * env;
    }

    const noise = this.ctx.createBufferSource();
    noise.buffer = buffer;

    const filter = this.ctx.createBiquadFilter();
    filter.type = isSpace ? 'lowpass' : 'bandpass';
    filter.frequency.setValueAtTime(
      isSpace ? 450 : 1200 + Math.random() * 500,
      now
    );
    filter.Q.setValueAtTime(isSpace ? 1.2 : 2.5, now);

    const gain = this.ctx.createGain();
    gain.gain.setValueAtTime(this.volume, now);

    noise.connect(filter);
    filter.connect(gain);
    gain.connect(this.ctx.destination);
    noise.start(now);
  }

  public playCarriageDing(): void {
    if (!this.enabled || !this.ctx) return;
    const now = this.ctx.currentTime;
    const osc = this.ctx.createOscillator();
    const gain = this.ctx.createGain();

    osc.type = 'sine';
    osc.frequency.setValueAtTime(2350, now); // Metal çan tınısı

    gain.gain.setValueAtTime(this.volume * 0.6, now);
    gain.gain.exponentialRampToValueAtTime(0.001, now + 0.65);
```

```
    osc.connect(gain);
    gain.connect(this.ctx.destination);
    osc.start(now);
    osc.stop(now + 0.65);
  }
}
```

5.3. Çekirdek Sınıf, Mürekkep Dağılması ve Overstrike Algoritması

```
// src/daktilo.ts
import { DaktiloOptions } from './types';
import { MechanicalSoundEngine } from './audio';

export class Daktilo {
  private elements: HTMLElement[];
  private opts: Required<DaktiloOptions>;
  private soundEngine: MechanicalSoundEngine;

  constructor(selector: string | HTMLElement | NodeListOf<HTMLElement>, options: DaktiloOptions = {}) {
    if (typeof selector === 'string') {
      this.elements = Array.from(document.querySelectorAll<HTMLElement>(selector));
    } else if (selector instanceof HTMLElement) {
      this.elements = [selector];
    } else {
      this.elements = Array.from(selector);
    }

    this.opts = {
      paper: options.paper ?? 'vintage',
      ribbonColor: options.ribbonColor ?? '#c84b31',
      maxWidth: options.maxWidth ?? '65ch',
      inkWear: options.inkWear ?? 0.35,
      animate: options.animate ?? false,
      speed: options.speed ?? 45,
      typoRate: options.typoRate ?? 0.02,
      overstrikeChar: options.overstrikeChar ?? 'x',
      sound: options.sound ?? false,
      volume: options.volume ?? 0.4,
      stamp: options.stamp ?? null,
    };

    this.soundEngine = new MechanicalSoundEngine(this.opts.sound, this.opts.volume);
    this.init();
  }

  private init(): void {
    this.injectGlobalStyles();
    this.elements.forEach((el) => {
      el.classList.add('daktilo-sheet', `daktilo-theme-${this.opts.paper}`);
      el.style.setProperty('--daktilo-ribbon', this.opts.ribbonColor);
      el.style.setProperty('--daktilo-width', this.opts.maxWidth);

      if (this.opts.stamp) this.renderStamp(el);

      if (this.opts.animate) {
        this.animateTypingWithOverstrike(el);
      } else if (this.opts.inkWear > 0) {
        this.applyOrganicInkWear(el);
      }
    });
  }

  // DOM yapısını bozmadan metin düğümlerine daktilo mürekkep kusuru uygular
  private applyOrganicInkWear(root: HTMLElement): void {
    const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT);
```

```
    const textNodes: Text[] = [];
    while (walker.nextNode()) textNodes.push(walker.currentNode as Text);

    textNodes.forEach((node) => {
      const text = node.nodeValue;
      if (!text || !text.trim()) return;

      const frag = document.createDocumentFragment();
      for (const char of text) {
        if (char === ' ' || Math.random() > this.opts.inkWear) {
          frag.appendChild(document.createTextNode(char));
        } else {
          const span = document.createElement('span');
          span.textContent = char;
          const yShift = ((Math.random() - 0.5) * 1.1).toFixed(2);
          const opacity = (0.72 + Math.random() * 0.28).toFixed(2);
          span.style.cssText = `display:inline-block;transform:translateY(${yShift}px);opacity:$
{opacity};`;
          frag.appendChild(span);
        }
      }
      node.parentNode?.replaceChild(frag, node);
    });
  }

  // Gerçekçi Overstrike (yanlış harfin üzerine X basma) destekli yazım motoru
  private async animateTypingWithOverstrike(el: HTMLElement): Promise<void> {
    const rawText = el.innerText;
    el.innerHTML = '';
    const contentSpan = document.createElement('span');
    const cursor = document.createElement('span');
    cursor.className = 'daktilo-cursor';
    cursor.textContent = '▉';
    el.append(contentSpan, cursor);

    const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
    const nearbyKeys = 'asdetrniolukm';

    for (let i = 0; i < rawText.length; i++) {
      const char = rawText[i];

      // Overstrike hatası simülasyonu
      if (char.match(/[a-zA-ZğüşıöçĞÜŞİÖÇ]/) && Math.random() < this.opts.typoRate) {
        const wrongChar = nearbyKeys[Math.floor(Math.random() * nearbyKeys.length)];
        const typoSpan = document.createElement('span');
        typoSpan.className = 'daktilo-overstrike';
        typoSpan.textContent = wrongChar;
        contentSpan.appendChild(typoSpan);
        this.soundEngine.playKeyClick(false);
        await sleep(this.opts.speed * 2.5); // Yazar hatayı fark eder

        typoSpan.setAttribute('data-strike', this.opts.overstrikeChar);
        this.soundEngine.playKeyClick(false);
        await sleep(this.opts.speed * 1.8);
      }

      contentSpan.appendChild(document.createTextNode(char));
      if (char === '
') {
        this.soundEngine.playCarriageDing();
        await sleep(this.opts.speed * 4);
      } else {
        this.soundEngine.playKeyClick(char === ' ');
        const jitter = this.opts.speed * (0.6 + Math.random() * 0.8);
        const punctuationPause = '.,;!?'.includes(char) ? this.opts.speed * 4 : 0;
        await sleep(jitter + punctuationPause);
      }
```

```
    }
  }

  private renderStamp(el: HTMLElement): void {
    const s = this.opts.stamp!;
    const header = document.createElement('div');
    header.className = 'daktilo-meta-header';
    header.innerHTML = `
      <span>KAYIT: ${s.docNo || '#001'} // TARİH: ${s.date || new Date().toLocaleDateString('tr-TR')}</
span>
      ${s.classification ? `<span class="daktilo-stamp-box">${s.classification}</span>` : ''}
    `;
    el.prepend(header);
  }

  private injectGlobalStyles(): void {
    if (typeof document === 'undefined' || document.getElementById('daktilo-core-css')) return;
    const style = document.createElement('style');
    style.id = 'daktilo-core-css';
    style.textContent = `
      @import url('https://fonts.googleapis.com/css2?
family=Courier+Prime:ital,wght@0,400;0,700;1,400&family=Special+Elite&display=swap');
      .daktilo-sheet {
        font-family: 'Courier Prime', monospace;
        max-width: var(--daktilo-width, 65ch);
        margin: 2rem auto;
        padding: 3rem 2.5rem;
        line-height: 1.75;
        position: relative;
        box-shadow: 0 4px 24px rgba(0,0,0,0.08);
      }
      .daktilo-theme-vintage {
        background-color: #f4f1ea;
        color: #262421;
        background-image: radial-gradient(rgba(0,0,0,0.035) 1px, transparent 0);
        background-size: 18px 18px;
      }
      .daktilo-sheet h1, .daktilo-sheet h2, .daktilo-sheet h3 {
        font-family: 'Special Elite', monospace;
        border-bottom: 2px dashed rgba(0,0,0,0.2);
        padding-bottom: 0.4rem;
      }
      .daktilo-sheet a {
        color: var(--daktilo-ribbon, #c84b31);
        text-decoration: none;
        border-bottom: 1px dashed var(--daktilo-ribbon, #c84b31);
      }
      .daktilo-overstrike {
        position: relative;
        display: inline-block;
        opacity: 0.75;
        margin-right: 1px;
      }
      .daktilo-overstrike::after {
        content: attr(data-strike);
        position: absolute;
        left: 0;
        top: -1px;
        color: var(--daktilo-ribbon, #c84b31);
        font-weight: bold;
      }
      .daktilo-meta-header {
        display: flex;
        justify-content: space-between;
        font-size: 0.8rem;
        border-bottom: 1px solid rgba(0,0,0,0.15);
        padding-bottom: 0.75rem;
```

```
        margin-bottom: 2rem;
        text-transform: uppercase;
        letter-spacing: 1px;
      }
      .daktilo-stamp-box {
        border: 2px solid var(--daktilo-ribbon, #c84b31);
        color: var(--daktilo-ribbon, #c84b31);
        padding: 1px 6px;
        font-weight: bold;
        transform: rotate(-2deg);
      }
      .daktilo-cursor {
        color: var(--daktilo-ribbon, #c84b31);
        animation: daktilo-blink 1s step-end infinite;
      }
      @keyframes daktilo-blink { 50% { opacity: 0; } }
    `;
    document.head.appendChild(style);
  }
}
```

6. PAKETLEME, DAĞITIM (BUILD PIPELINE) VE ENTEGRASYON

Kütüphanenin hem modern bundler'larla (Vite, Webpack, Next.js) hem de doğrudan HTML `<script>` etiketiyle (CDN üzerinden) kullanılabilmesi için **tsup** (esbuild tabanlı) kullanılacaktır.

```
// package.json
{
  "name": "daktilo-js",
  "version": "1.0.0",
  "description": "Zero-dependency typewriter paper aesthetic, organic ink wear & Web Audio mechanical sound engine.",
  "keywords": ["typewriter", "retro", "typography", "theme-engine", "webaudio", "paper-ui"],
  "license": "MIT",
  "main": "./dist/index.cjs",
  "module": "./dist/index.mjs",
  "unpkg": "./dist/index.global.js",
  "jsdelivr": "./dist/index.global.js",
  "types": "./dist/index.d.ts",
  "files": ["dist"],
  "scripts": {
    "build": "tsup src/index.ts --format cjs,esm,iife --global-name Daktilo --dts --minify",
    "dev": "tsup src/index.ts --format esm,iife --global-name Daktilo --watch",
    "test": "vitest run"
  },
  "devDependencies": {
    "tsup": "^8.3.0",
    "typescript": "^5.6.0",
    "vitest": "^2.1.0"
  }
}
```

6.1. Farklı Ortamlarda Kullanım Örnekleri

1. VANILLA HTML / CDN (SIFIR KURULUM)

Klasik bir blog veya statik sayfaya tek satırda entegrasyon:

```
<article id="my-essay">
  <h1>Gece Notları</h1>
```

```
  <p>Mekanik tuşların kağıt üzerindeki izi, dijital ekranın soğukluğunu kırar...</p>
</article>

<script src="https://unpkg.com/daktilo-js@1.0.0/dist/index.global.js"></script>
<script>
  new Daktilo.Daktilo('#my-essay', {
    paper: 'vintage',
    inkWear: 0.4,
    sound: true,
    stamp: { docNo: '#84-A', classification: 'ARŞİV' }
  });
</script>
```

2. REACT / NEXT.JS HOOK KULLANIMI

Modern SPA/SSR uygulamalarında Custom Hook ile kullanım:

```
import { useEffect, useRef } from 'react';
import { Daktilo, DaktiloOptions } from 'daktilo-js';

export function useDaktilo<T extends HTMLElement>(options?: DaktiloOptions) {
  const ref = useRef<T | null>(null);
  useEffect(() => {
    if (!ref.current) return;
    const instance = new Daktilo(ref.current, options);
    return () => { /* Cleanup if needed */ };
  }, []);
  return ref;
}
```

7. FONKSIYONEL OLMAYAN GEREKSINIMLER (PERFORMANS & ERIŞILEBILIRLIK)

- **Erişilebilirlik (a11y & Screen Readers):** `applyOrganicInkWear` fonksiyonu harfleri `<span>` içerisine aldığında ekran okuyucuların kelime bütünlüğünü kaybetmemesi için ana paragrafa `aria-label` korunumu uygulanmalı veya `prefers-reduced-motion: reduce` aktif olan kullanıcılarda animasyon ve ses otomatik devre dışı bırakılmalıdır.
- **Performans Sınırı (DOM Node Cap):** 10.000+ kelimelik çok uzun makalelerde her harfi `<span>` içine almak DOM düğüm sayısını şişirebilir. Bu nedenle `inkWear` algoritması yalnızca kusur uygulanacak karakterleri (varsayılan %25-35) `<span>` içine alır, geri kalan düz metinleri tek bir `TextNode` olarak birleştirir.
- **Tarayıcı Otomatik Oynatma (Autoplay) Politikası:** Chrome ve Safari, kullanıcı etkileşimi olmadan `AudioContext` başlatılmasına izin vermez. Ses motoru ilk `pointerdown` veya `keydown` olayında sessizce kilidi açacak şekilde tasarlanmıştır.

8. ÜRÜN YOL HARITASI (ROADMAP) VE BAŞARI METRIKLERI

FAZ	HEDEF SÜRE	TESLIMATLAR (DELIVERABLES)
Faz 1: MVP (v1.0)	Hafta 1 - 2	Çekirdek TypeScript sınıfı, 3 kağıt teması (`vintage`, `clean`, `carbon`), Web Audio ses sentezleyici, Overstrike animasyonu ve NPM/CDN yayını.
	Hafta 3 - 4	İnteraktif Playground / Demo web sitesi (kullanıcıların slider ile mürekkep dağılmasını test edip konfigürasyon

FAZ	HEDEF SÜRE	TESLIMATLAR (DELIVERABLES)
Faz 2: Ekosistem (v1.2)		kopyalayabildiği sayfa), React (`<DaktiloSheet />`) ve Astro bileşen sarmalayıcıları.
Faz 3: Eklentiler (v1.5)	Ay 2	Canlı Editör Modu (`<textarea>` veya `contenteditable` alanlarını gerçek zamanlı sesli daktiloya çeviren Yazar Modu) ve Tailwind CSS eklentisi (`tailwindcss-daktilo`).

AÇIK KAYNAK BÜYÜME VE LANSMAN STRATEJISI (GO-TO-MARKET)

Lansman günü **GitHub Pages / Vercel** üzerinde canlı bir "Daktilo Playground" yayına alınacaktır. Ziyaretçiler kendi metinlerini yazarken mekanik tuş ve satır sonu "ding" sesini deneyimleyebilecek, tek tıkla `npm i daktilo-js` kodunu kopyalayabilecektir. Bu görsel ve işitsel deneyim; **Product Hunt**, **Hacker News (Show HN)** ve **Reddit (r/javascript, r/webdev)** topluluklarında yüksek etkileşim üretmek üzere kurgulanmıştır.